Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Arquitectura de Computadores y Ensambladores 2 A
Ing. Gabriel Alejandro Díaz
Auxiliar 1: Josue Antonio Cux Barrientos
Auxiliar 2: Marcos Geovani Josías Pérez Secay

Proyecto 1

SmartEgg: Plataforma IoT Distribuida

MQTT, API REST, Grafana y Visualización de Datos Avanzada

PONDERACIÓN: 20 pts

Tiempo estimado: 45 hrs

Continuación directa de la Práctica 1 — SmartEgg: Incubadora Inteligente IoT

Índice

Índice 2
1. Marco Formativo 4
    1.1. Valor 4
    1.2. Competencia(s) 4
    1.3. Habilidad(es) blandas a formar 4
2. Resultado del Aprendizaje 4
    2.1. Objetivo SMART 4
3. Resumen 5
4. Enunciado del Proyecto 6
    4.1 Continuidad respecto a la Práctica 1 6
    4.2 Descripción del problema o necesidad a resolver 6
    4.3 Arquitectura General 6
        Descripción de cada conexión del flujo 8
    4.4 Alcance del proyecto 8
5. Backend: API REST y Cliente MQTT 9
        5.1 Requerimientos funcionales 9
        5.2 Consideraciones técnicas 9
6. Broker MQTT 9
        6.1 Jerarquía de topics sugerida 9
        6.2 Requerimientos funcionales 10
        6.3 Consideraciones 10
7. Base de Datos y Persistencia 10
        7.1 Estructura mínima recomendada 10
        7.2 Requerimientos funcionales 10
8. Frontend y Alarmas en Tiempo Real 11
        8.1 Requerimientos funcionales 11
        8.2 Consideraciones 11
9. Grafana: Monitoreo en Tiempo Real y Analítica Histórica 11
        9.1 Paneles en tiempo real (fuente: broker MQTT) 11
        9.2 Paneles históricos (fuente: base de datos) 11
        9.3 Requerimientos funcionales 11
10. Visualización de Datos Avanzada 12
        10.1 Frameworks y librerías de visualización de datos 12
        10.2 Visualización interactiva con p5.js y Three.js 12
        10.3 Video demostrativo 12
        10.4 Requerimientos funcionales 12
11. HTTP vs. MQTT en IoT 12
        11.1 Guía de investigación 13
        11.2 Entregable 13
12. Orquestación con Docker 13
        12.1 Requerimientos funcionales 13
        12.2 Consideraciones 13
13. Repositorio de GitHub 13

14. Restricciones 14
15. Documentación 14
16. Entregables 14
17. Material de Apoyo 15
18. Recursos y Herramientas a Utilizar 16
19. Cronograma 16
20. Rúbrica de Calificación 16
    20.1 Requisitos para optar a la calificación 17
    20.2 Resumen de Puntuaciones 17
    20.3 Detalle de la Calificación 17
    20.4 Comentarios Generales 23

1. Marco Formativo

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Innovación y responsabilidad técnica	Se aplica al diseñar una arquitectura distribuida completa (backend, broker MQTT, base de datos, dashboards y frontend) que debe operar de forma coherente, segura y trazable, construyendo sobre el nodo físico validado en la Práctica 1.

1.2. Competencia(s)

Competencia General:

- Aplica principios de ingeniería, ciencias de la computación y sistemas de información y comunicación en la formulación y resolución de problemas complejos de arquitectura distribuida.

Competencias Específicas:

- Diseña e implementa arquitecturas IoT distribuidas que integran comunicación serial, protocolos publish/subscribe (MQTT) y servicios REST.
- Desarrolla soluciones de persistencia de datos y las conecta con herramientas de observabilidad (Grafana) para monitoreo en tiempo real e histórico.
- Construye visualizaciones de datos interactivas en la web utilizando librerías y frameworks especializados, incluyendo gráficos 2D/3D.
- Compara y selecciona protocolos de comunicación (HTTP vs. MQTT) según el contexto de un sistema IoT.
- Orquesta múltiples servicios de la solución utilizando contenedores Docker.

1.3. Habilidad(es) blandas a formar

- **Comunicación efectiva:** presentación clara de decisiones de arquitectura, avances y resultados técnicos durante el desarrollo e integración del proyecto.
- **Resolución de problemas:** análisis y solución de fallos relacionados con la comunicación serial, MQTT, persistencia de datos y sincronización entre los distintos servicios.
- **Pensamiento crítico:** evaluación de decisiones técnicas de diseño distribuido, seguridad de comunicaciones y selección de protocolos según el contexto (HTTP vs. MQTT).
- **Trabajo colaborativo:** coordinación efectiva en equipo para integrar componentes desarrollados en paralelo (hardware, backend, broker, frontend, dashboards).
- **Gestión del tiempo:** planificación de un proyecto multi-componente con dependencias técnicas entre subsistemas.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Diseñar e implementar una plataforma IoT distribuida para SmartEgg que integre el nodo físico de la Práctica 1 con un backend REST/MQTT, un broker de mensajería, persistencia relacional, dashboards en Grafana y un frontend con alarmas en tiempo real, incorporando además

	visualización de datos avanzada (p5.js/Three.js), un análisis comparativo HTTP vs. MQTT y orquestación de servicios con Docker.
Medible (¿Cuánto?)	El sistema deberá publicar el 100% de las lecturas del nodo físico al broker MQTT, reflejar alarmas en el frontend en menos de 3 segundos desde su publicación, exponer al menos 6 endpoints REST documentados y desplegar la totalidad de los servicios (backend, broker, base de datos, Grafana) mediante un único docker-compose funcional.
Alcanzable (¿Cómo?)	Se logrará utilizando un backend que lea el puerto serial del microcontrolador, publique/consuma mensajes MQTT mediante un broker (Mosquitto o HiveMQ), persista datos en una base de datos relacional, y exponga dashboards en Grafana conectados tanto al broker (tiempo real) como a la base de datos (históricos).
Realista (¿Para qué?)	Permite consolidar el dominio de arquitecturas IoT distribuidas de nivel productivo, replicando patrones utilizados en la industria para telemetría, monitoreo remoto y observabilidad de sistemas embebidos.
A Tiempo (¿Cuándo?)	Deberá completarse dentro del período definido para el proyecto en el segundo semestre de 2026.

3. Resumen

Este proyecto es la segunda fase (Practica 1) del sistema SmartEgg iniciado en la Práctica 1. Mientras que en la fase anterior el objetivo fue construir y validar el nodo físico de la incubadora (sensores, actuadores, LCD y control por umbrales, transmitiendo datos únicamente por puerto serial hacia la computadora), en esta fase se debe transformar ese nodo físico en una plataforma IoT distribuida, completa y observable.

Se conserva la lectura de sensores, el control de actuadores y la comunicación serial UART/USB. Lo que cambia es todo lo que ocurre después del puerto serial. Un backend leerá continuamente los datos entregados por el microcontrolador y cumplirá una doble función: exponer una API REST para el frontend y la base de datos, y actuar como cliente MQTT publicando la telemetría hacia un broker. Ese broker centraliza la comunicación en tiempo real: por un lado alimenta paneles de Grafana con datos en vivo, y por otro entrega alarmas al frontend mediante una suscripción (típicamente sobre WebSocket). La base de datos relacional conserva el histórico y alimenta consultas estáticas adicionales dentro de Grafana.

Adicionalmente, el proyecto incorpora cuatro componentes formativos nuevos, orientados a ampliar el dominio de tecnologías IoT y de visualización de datos que un ingeniero en sistemas debe manejar:

- **Visualización de datos avanzada:** uso de frameworks de graficación web y librerías de consumo de APIs, incluyendo visualizaciones interactivas 2D/3D con p5.js y Three.js.
- **Comparación HTTP vs. MQTT:** análisis técnico de ambos modelos de comunicación en contextos IoT, incluyendo optimizaciones de HTTP para dispositivos restringidos.
- **Orquestación con Docker:** despliegue de todos los servicios de la plataforma (broker, backend, base de datos, Grafana) mediante contenedores coordinados.
- **Observabilidad con Grafana:** dashboards en tiempo real (vía broker) y analítica histórica (vía base de datos).

Con este proyecto el estudiante consolida una arquitectura IoT de extremo a extremo: desde el sensor físico hasta el dashboard y la alarma visual en el navegador, pasando por los patrones de comunicación (serial, REST, publish/subscribe) que sostienen a la mayoría de soluciones IoT productivas.

4. Enunciado del Proyecto

4.1 Continuidad respecto a la Práctica 1

La Facultad de Ciencias Agronómicas de la USAC quedó satisfecha con el prototipo físico de SmartEgg entregado en la Práctica 1 y ahora solicita evolucionar la solución hacia un servicio IoT completo ("SmartEgg as a Service") que pueda ofrecerse a distintas granjas avícolas de forma remota. El nodo físico validado (microcontrolador + sensores + actuadores + LCD, comunicándose por serial) se mantiene sin cambios estructurales; el reto de este proyecto es construir, alrededor de ese nodo, la infraestructura de software distribuida que permita monitorear, controlar y visualizar el proceso de incubación desde cualquier dispositivo.

Importante: todo el hardware, sensores, actuadores y la lógica de control por umbrales desarrollada en la Práctica 1 se reutiliza. El microcontrolador continúa comunicándose exclusivamente por UART/USB con la computadora; no se agregan módulos WiFi/Bluetooth al nodo físico en esta fase.

4.2 Descripción del problema o necesidad a resolver

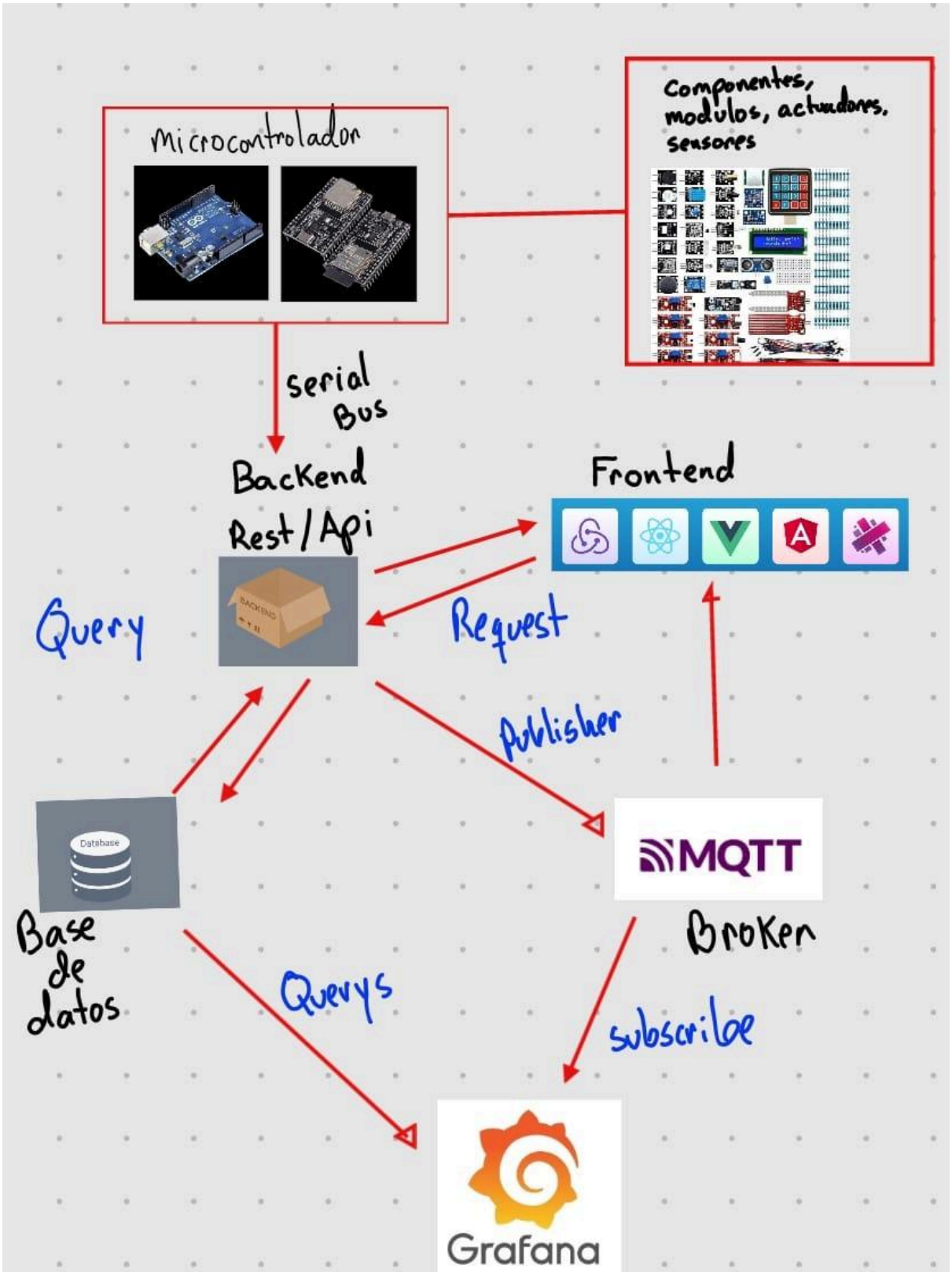
Aunque el nodo físico de SmartEgg funciona correctamente de forma aislada, los avicultores necesitan supervisar múltiples incubadoras de manera remota, recibir alarmas inmediatas ante condiciones críticas, consultar el historial de temperatura y humedad, y tomar decisiones basadas en datos. Un sistema que solo imprime datos por el puerto serial de una computadora no resuelve estas necesidades.

Se requiere entonces una capa de software que centralice la información de uno o varios nodos SmartEgg, la distribuya en tiempo real a los interesados (dashboards, alarmas) y la conserve de forma histórica para análisis posterior, todo con las buenas prácticas de una arquitectura IoT distribuida basada en el patrón publish/subscribe.

Se agregara una nueva sección debido a que varios avicultores les gusto la incubadora, muchos empiezan a dejar otros huevos y se llevan los que ya están, haciendo que los principales tenga dificultad con el registro de sus propios huevos y alterando el sistema por lo cual se deben de registrar a dos personas encargadas, lo cual solo ellos pueden abrir y cerrar la incubadora mediante un registro ya sea por huella, tarjeta o llave, las cargos son Operador 1 y Operador 2 con esto se espera poder solucionar el problema que ellos tienen.

4.3 Arquitectura General

La arquitectura general del proyecto conserva el nodo físico de la Práctica 1 sin modificaciones y añade una capa de aplicación distribuida. El siguiente diagrama resume el flujo de datos obligatorio para este proyecto:


Microcontrolador
Componentes, modulos, actuadores, sensores
serial Bus
BackEnd Rest/Api
Frontend
Query
Request
Publisher
Database
MQTT
Broker
Base de datos
Querys
subscribe
Grafana

Descripción de cada conexión del flujo

Conexión	Descripción
Sensores/Actuadores → Microcontrolador	Sin cambios respecto a la Práctica 1: lectura de DHT, control de relé, servomotor y buzzer por parte del Arduino/ESP32.
Microcontrolador → PC (Serial UART/USB)	El nodo físico continúa transmitiendo lecturas y eventos exclusivamente por comunicación serial hacia la computadora que ejecuta el backend.
Backend → Frontend y Base de Datos	El backend expone una API REST consumida por el frontend, y persiste/consulta información en la base de datos relacional.
Backend (cliente MQTT) → Broker	El backend pública en el broker MQTT los datos leídos del puerto serial (telemetría) y eventos relevantes (alarmas, cambios de estado).
Broker → Grafana	Grafana se suscribe al broker (o a un datasource compatible) para graficar telemetría en tiempo real.
Broker → Frontend (alarmas)	El frontend se suscribe al broker (por ejemplo mediante MQTT sobre WebSocket) para recibir y mostrar alarmas en tiempo real sin necesidad de refrescar la página.
Base de Datos → Grafana	Grafana consulta directamente la base de datos relacional para construir paneles de analítica histórica (queries estáticos: promedios, máximos/mínimos, tendencias).

Esta arquitectura es de cumplimiento obligatorio en su estructura general (los cinco componentes: backend, broker, base de datos, Grafana y frontend, con las conexiones indicadas).

La tecnología específica utilizada en cada componente (framework de backend, motor de base de datos, broker MQTT, framework de frontend) queda a discreción del estudiante, salvo lo indicado en la sección de Restricciones.

4.4 Alcance del proyecto

Alcance obligatorio:

- Conservar y reutilizar el nodo físico validado en la Práctica 1, sin modificar su lógica de control por umbrales.
- Implementar un backend que lea el puerto serial y exponga una API REST funcional.
- Implementar un cliente MQTT en el backend que publique telemetría y eventos hacia un broker.
- Desplegar y configurar un broker MQTT con una jerarquía de topics documentada.
- Diseñar e implementar una base de datos relacional con dimensión temporal para conservar el historial.
- Construir un frontend web (o web/móvil) que consuma la API REST y reciba alarmas en tiempo real vía el broker.
- Configurar dashboards en Grafana: al menos uno alimentado en tiempo real por el broker y al menos uno de analítica histórica alimentado por la base de datos.
- Incorporar al menos una visualización de datos interactiva construida con p5.js o Three.js.
- Elaborar el documento comparativo HTTP vs. MQTT en el contexto de este proyecto.
- Orquestar los servicios de la solución (broker, backend, base de datos, Grafana) con Docker mediante un docker-compose funcional.

5. Backend: API REST y Cliente MQTT

El backend es el componente central de la arquitectura: es el único proceso que se comunica directamente con el nodo físico (vía puerto serial) y es responsable de distribuir esa información hacia el resto del sistema.

6. Broker MQTT

El broker MQTT es el punto de intercambio de mensajes en tiempo real entre el backend, Grafana y el frontend. Se recomienda utilizar Mosquitto o HiveMQ, aunque se permite cualquier broker MQTT estándar (v3.1.1 o v5).

6.1 Jerarquía de topics sugerida

Topic	Publicador	Suscriptor(es)	Descripción
smartegg/{id}/sensores/temperatura	Backend	Grafana	Última lectura de temperatura del nodo.
smartegg/{id}/actuadores/estado	Backend	Grafana, Frontend	Estado actual de calefacción, ventilación y rotación.
smartegg/{id}/alarmas/critica	Backend	Frontend	Evento de alarma ante condición fuera de rango (retained + QoS 1).
smartegg/{id}/status	Backend (LWT)	Frontend, Grafana	Disponibilidad del nodo (online/offline) mediante Last Will and Testament.

El identificador {id} permite escalar el diseño a múltiples incubadoras sin cambiar la estructura de topics (alcance opcional de múltiples nodos).

6.2 Requerimientos funcionales

- Definir y documentar en el README la jerarquía completa de topics utilizada, incluyendo el formato (JSON) de cada payload.
- Configurar el nivel de QoS adecuado para cada topic (se recomienda QoS 1 como mínimo para alarmas).
- Configurar al menos un mensaje retained (por ejemplo, el último estado conocido del nodo) y un Last Will and Testament (LWT) para notificar desconexiones inesperadas.
- Verificar mediante un cliente MQTT de prueba (MQTT Explorer, MQTTX, mosquitto_sub) que los topics publican y se reciben correctamente, incluyendo evidencia en la documentación.

6.3 Consideraciones

- El broker puede ejecutarse localmente, en un contenedor Docker (ver sección 12) o en un servicio administrado (por ejemplo HiveMQ Cloud), siempre que el proyecto documente claramente cómo acceder a él.
- Evitar publicar payloads pesados o innecesarios: MQTT está optimizado para mensajes pequeños y frecuentes.

7. Base de Datos

La base de datos relacional conserva el historial completo del sistema y es la fuente de los paneles de analítica histórica en Grafana. El motor (PostgreSQL, MySQL, SQLite, SQL Server, etc.) queda a discreción del estudiante.

También se debe tomar en cuenta el registro de los operadores.

7.1 Estructura mínima recomendada

- **Lecturas:** historial de temperatura, humedad y espacios disponibles, con marca de tiempo por día.
- **Eventos/Alarmas:** registro de condiciones críticas, con tipo, severidad, timestamp de inicio y de resolución.
- **Lotes/Huevos:** ficha de identificación del huevo o lote ingresado, fecha de ingreso, y su relación con el nodo correspondiente.
- **Operador:** nombre del operador, id de operador, edad y género
- **Apertura de la puerta:** día, hora, id_operador, tipo_abierto, tipo_cerrado o Tipo_abierto, cantidad de huevos que ingreso o retiro.

7.2 Requerimientos funcionales

- Diseñar el modelo entidad-relación considerando la dimensión temporal.
- Implementar las consultas necesarias para alimentar los paneles históricos de Grafana (mínimo, máximo y promedio por rango de tiempo).
- Documentar el modelo con una herramienta como dbdiagram.io o similar.

8. Frontend y Alarmas en Tiempo Real

El frontend hereda y mejora la vista visual solicitada en la Práctica 1 (visualización de sensores, ficha de identificación del huevo, generación de reportes PDF) y añade la capacidad de recibir alarmas en tiempo real directamente desde el broker MQTT.

Debe de existir un perfil para cada operador donde ellos pueden consultar sus propios diagnósticos y sus informes generales.

Debe de haber una implementación de usuario administrador.

8.1 Requerimientos funcionales

- Consumir la API REST del backend para mostrar el estado actual y el historial del sistema.
- Suscribirse al broker MQTT (típicamente mediante MQTT sobre WebSocket, por ejemplo con la librería MQTT.js) para recibir alarmas y reflejarlas visualmente sin recargar la página.
- Mostrar de forma clara y distinguible las alarmas activas (color, ícono, sonido, notificación tipo toast/banner).
- Mantener el formulario de registro/ficha de identificación de huevos/lotes y la generación y descarga de reportes en PDF, mejorando lo entregado en la Práctica 1.
- Implementar nuevas soluciones solicitadas.
- Ser responsivo: debe poder visualizarse correctamente desde un dispositivo móvil o tablet.
- Implementar de P5.js

8.2 Consideraciones

- El framework de frontend es libre (React, Vue, Angular, Svelte, etc.).
- Se recomienda separar claramente el cliente MQTT (para alarmas) del cliente HTTP/REST (para datos generales), documentando ambos flujos.
- Validar y controlar errores en los formularios, igual que en la Práctica 1.

9. Grafana: Monitoreo en Tiempo Real y Analítica Histórica



Grafana centraliza la observabilidad del sistema, combinando dos fuentes de datos claramente diferenciadas: el broker MQTT (tiempo real) y la base de datos relacional (consultas estáticas/históricas).

9.1 Paneles en tiempo real (fuente: broker MQTT)

- Temperatura y humedad actuales, actualizadas en vivo conforme se publican en el broker.
- Estado actual de los actuadores (calefacción, ventilación, rotación).
- Indicador de disponibilidad del nodo (online/offline) basado en el topic de status/LWT.

9.2 Paneles históricos (fuente: base de datos)

- Tendencia de temperatura y humedad en rangos de tiempo seleccionables (última hora, 24 horas, última semana).
- Mínimo, máximo y promedio de temperatura y humedad por lote/huevo registrado.
- Historial de alarmas, incluyendo duración de cada condición crítica.
- Cantidad de veces abiertas la puerta
- Cantidad de veces cerrada la puerta

10. Visualización de Datos Avanzada

Además del dashboard funcional del frontend, el proyecto busca que el estudiante amplíe su dominio de herramientas de visualización de datos en la web, aplicándolas sobre los datos reales generados por SmartEgg (temperatura, humedad, rotación, ocupación de espacios, alarmas y etc).

10.1 Frameworks y librerías de visualización de datos

Se debe integrar al menos un framework/librería de visualización orientado a gráficos (líneas, barras, gauges, etc.) que consuma datos reales desde la API REST del backend o desde el broker MQTT. Ejemplos aceptados: Chart.js, D3.js, Plotly.js, Recharts, ECharts, entre otros.

- Investigar y justificar la elección del framework utilizado (ventajas, curva de aprendizaje, tipo de gráficos que ofrece).
- Consumir los datos mediante una librería de peticiones HTTP (fetch, Axios) o un cliente MQTT sobre WebSocket, según corresponda al gráfico.

10.2 Visualización interactiva con p5.js y Three.js

Se debe construir al menos una visualización interactiva (2D con p5.js o 3D con Three.js) que represente de forma creativa el estado del sistema SmartEgg. Algunos ejemplos orientadores (no obligatorios, el diseño final queda a discreción del estudiante):

- Una representación 3D de la incubadora con Three.js donde el color de la escena cambie según la temperatura y un objeto gire para representar la rotación periódica de los huevos.
- Una visualización 2D con p5.js donde partículas o formas reaccionen en tiempo real a la humedad y temperatura recibidas.
- Un medidor (gauge) o mapa de calor interactivo construido a mano con p5.js a partir de datos históricos.

Esta visualización debe consumir datos reales del sistema (vía REST o MQTT), no datos simulados o estáticos.

10.3 Video demostrativo

Se debe grabar y entregar un video corto (2-4 minutos) donde se demuestre la interacción con la visualización desarrollada (p5.js/Three.js), explicando qué representa cada elemento visual y cómo reacciona a los datos del sistema en tiempo real.

10.4 Requerimientos funcionales

- Documentar en el README qué framework(s) de visualización se utilizaron y cómo consumen los datos (API/librería específica).
- Incluir el enlace o archivo del video demostrativo dentro del repositorio o la documentación.
- La visualización interactiva debe ser accesible desde el frontend del proyecto (integrada o enlazada).

11. HTTP vs. MQTT en IoT

Este componente es una investigación dirigida que se documenta y sustenta, y que debe reflejarse en las decisiones de arquitectura tomadas dentro del propio proyecto.

11.1 Guía de investigación

- Explicar el modelo de comunicación de HTTP (petición/respuesta) y su uso típico en soluciones IoT (por ejemplo, envío periódico de datos mediante peticiones POST).
- Describir las optimizaciones que existen para el uso de HTTP en contextos IoT: keep-alive, compresión de payloads, HTTP/2 (multiplexado), caching, agrupación de lecturas en lotes (batching), y protocolos derivados como CoAP diseñados para dispositivos restringidos.
- Explicar el modelo publish/subscribe de MQTT: broker, topics, QoS, retained messages y Last Will and Testament.
- Comparar ambos protocolos según: modelo de comunicación, overhead de cabeceras, consumo de energía/ancho de banda, comportamiento ante desconexiones, y escalabilidad ante múltiples dispositivos.

- Justificar, con base en la comparación anterior, por qué este proyecto utiliza MQTT para la telemetría en tiempo real y REST/HTTP para las operaciones bajo demanda del frontend (CRUD, consultas puntuales).

11.2 Entregable

El resultado de esta investigación debe integrarse como una sección específica dentro del documento técnico (README), incluyendo al menos una tabla comparativa entre HTTP y MQTT construida por el propio estudiante.

12. Orquestación con Docker

Para facilitar el despliegue y evidenciar buenas prácticas de arquitectura distribuida, la solución debe orquestar mediante Docker o Podman.

12.1 Requerimientos funcionales

- Crear un archivo docker-compose.yml que levante, como mínimo: el broker MQTT, el backend y la base de datos. Se recomienda incluir también Grafana.
- Cada servicio debe ejecutarse en su propio contenedor, comunicándose mediante una red Docker definida en el compose.
- Utilizar volúmenes para persistir los datos de la base de datos y, si aplica, la configuración de Grafana.
- Externalizar la configuración sensible (credenciales, puertos, URLs) mediante variables de entorno (archivo .env).
- Documentar en el README los comandos necesarios para levantar toda la plataforma (por ejemplo docker compose up -d) y verificar que cada servicio está operativo.

13. Repositorio de GitHub

Todo el código, la documentación y los archivos de orquestación deberán subirse al mismo repositorio utilizado en la Práctica 1, dentro de una nueva carpeta Proyecto_1.

- Cada integrante del grupo debe realizar al menos 3 commits durante el desarrollo del proyecto.
- Todo código o documento que no se encuentre en el repositorio, o que sea subido después de la fecha de entrega, no será tomado en cuenta para la calificación.

14. Restricciones

- Nodo físico: se debe reutilizar el nodo físico de la Práctica 1 sin alterar su lógica de control por umbrales. (-100%)
- Arquitectura: el proyecto debe implementar los cinco componentes obligatorios (backend, broker MQTT, base de datos, Grafana y frontend) con las conexiones descritas en la sección 4.3. (-100%)
- Broker MQTT: el proyecto debe utilizar un broker MQTT real y funcional; no se acepta simular la comunicación en tiempo real únicamente por HTTP/polling. (-100%)
- Docker: la orquestación mínima (broker, backend y base de datos) debe estar contenida en un docker-compose funcional. (-10%)
- Documentación: la documentación debe estar completa o se recibe una penalización del 20%.

- Entrega tarde: se deberán enviar todos los entregables y commits en la fecha establecida; de no ser así se tendrá una penalización del 50%.

15. Documentación

La documentación deberá realizarse en formato Markdown en el README del repositorio, dentro de la carpeta Proyecto_1. Se solicita:

- Descripción de la solución y continuidad respecto a la Práctica 1.
- Diagrama de arquitectura general (igual o similar al presentado en la sección 4.3), explicando cada conexión.
- Documentación de la jerarquía de topics MQTT (nombre, publicador, suscriptores, formato del payload, QoS).
- Modelo entidad-relación de la base de datos.
- API Contracts del backend (Postman, Insomnia, Swagger/OpenAPI, etc.).
- Capturas de los dashboards de Grafana (tiempo real e históricos).
- Explicación de los frameworks/librerías utilizados para la visualización avanzada de datos (p5.js/Three.js y librería de gráficos), incluyendo el enlace al video demostrativo.
- Documento comparativo HTTP vs. MQTT solicitado en la sección 11.
- Instrucciones y evidencia de la orquestación con Docker (docker-compose y capturas de los contenedores en ejecución).
- Capturas de GitHub evidenciando los commits del equipo.
- Manual de usuario del sistema completo.

16. Entregables

Tipo	Descripción
Código fuente	Backend, configuración del broker, scripts de base de datos, frontend y las visualizaciones avanzadas (p5.js/Three.js), organizados y comentados.
Documento técnico	Informe en Markdown (README) según lo solicitado en la sección de Documentación.
Video demostrativo	Video de 2 a 4 minutos mostrando la visualización interactiva (p5.js/Three.js) reaccionando a datos reales del sistema.
Orquestación Docker	Archivo(s) docker-compose y configuración necesaria para levantar la plataforma.
Evidencias de funcionamiento	Capturas de Grafana, del broker recibiendo/publicando mensajes, del frontend recibiendo alarmas en tiempo real, y del sistema físico operando.
Repositorio Git	Repositorio organizado con código fuente, documentación y evidencias, con commits progresivos por integrante.

17. Material de Apoyo

Categoría	Recurso	Descripción
Documentación oficial	mqtt.org	Conceptos fundamentales del protocolo MQTT y arquitectura publish/subscribe.
Documentación oficial	mosquitto.org	Configuración y uso del broker MQTT Mosquitto.
Documentación oficial	HiveMQ MQTT Essentials	Guía completa de topics, QoS, retained messages y LWT.
Documentación oficial	Grafana Docs	Configuración de datasources, dashboards y alerting en Grafana.
Documentación oficial	Docker Docs	Uso de Docker y Docker Compose para orquestación de servicios.
Documentación oficial	p5.js Reference	Referencia oficial de la librería p5.js para visualización 2D interactiva.
Documentación oficial	Three.js Docs	Documentación oficial de Three.js para visualización 3D en la web.
Documentación oficial	Chart.js Docs	Referencia de un framework popular de visualización de datos en gráficos.
Documentación oficial	MDN - HTTP	Referencia del protocolo HTTP: métodos, cabeceras y optimizaciones.
Manual	Random Nerd Tutorials	Tutoriales prácticos para ESP32/Arduino, sensores y proyectos IoT.
Ejemplo	MQTT Explorer	Cliente gráfico para inspeccionar topics y mensajes MQTT durante el desarrollo.
Ejemplo	dbdiagram.io	Herramienta para diagramar el modelo entidad-relación de la base de datos.

18. Recursos y Herramientas a Utilizar

Tipo (Obligatorio/Opcional)	Categoría	Descripción
Obligatorio	Hardware	Nodo físico SmartEgg (Arduino/ESP32) desarrollado en la Práctica 1, sin modificaciones estructurales.
Obligatorio	Software - Backend	Framework de backend a elección (Node.js/Express, Spring Boot, Django, FastAPI, etc.) con soporte de cliente MQTT y servidor REST.
Obligatorio	Plataforma	Broker MQTT (Mosquitto, HiveMQ, EMQX, etc.).
Obligatorio	Base de datos	Motor relacional a elección (PostgreSQL, MySQL, SQL Server, SQLite, etc.).
Obligatorio	Plataforma	Grafana para dashboards en tiempo real e históricos.
Obligatorio	Software - Frontend	Framework de frontend a elección (React, Vue, Angular, Svelte, etc.).

Tipo (Obligatorio/Opcional)	Categoría	Descripción
Obligatorio	Visualización	p5.js o Three.js para la visualización interactiva, y una librería de gráficos (Chart.js, D3.js, Plotly, etc.).
Obligatorio	Plataforma	Docker y Docker Compose para la orquestación de servicios.
Obligatorio	Control de versiones	GitHub para control de versiones y entrega del proyecto.
Opcional	Herramientas de prueba	MQTT Explorer, MQTTX o mosquitto_pub/sub para validar la comunicación MQTT.

19. Cronograma

Actividad	Fecha Inicio	Fecha Fin
Asignación del proyecto	16/08/2026	12/09/2026
Diseño de arquitectura, topics MQTT y modelo de datos	Semana B	Semana B
Implementación de backend, broker y base de datos	Semana C	Semana D
Implementación de frontend, alarmas y dashboards de Grafana	Semana D	Semana E
Visualización avanzada (p5.js/Three.js) y video demostrativo	Semana E	Semana F
Orquestación con Docker, pruebas y documentación	Semana F	Semana G
Entrega final	12/09/2026	12/09/2026
Calificación	12/09/2026	19/09/2026

20. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación del proyecto, lo cual permite claridad para el estudiante y el tutor académico sobre los aspectos a evaluar.

20.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, se debe cumplir con los requisitos que se indican en esta sección; de no cumplirse, la nota del proyecto tendrá un valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
Continuidad del nodo físico	El nodo físico de la Práctica 1 opera correctamente y conserva su lógica de control por umbrales.	
Arquitectura completa	Los cinco componentes obligatorios (backend, broker MQTT, base de datos, Grafana, frontend) están implementados y conectados según lo solicitado.	

Tema	Descripción	Cumple (Sí/No)
Comunicación MQTT funcional	El backend publica y el broker distribuye mensajes correctamente hacia Grafana y el frontend.	
Orquestación Docker	Existe un docker-compose funcional que levanta como mínimo broker, backend y base de datos.	
Documentación obligatoria	Se entrega el documento técnico completo según lo solicitado en la sección de Documentación.	

20.2 Resumen de Puntuaciones

Área	Puntos Totales
1. Continuidad y Capa Física	5
2. Backend - API REST y Cliente MQTT	15
3. Broker MQTT	10
4. Base de Datos y Persistencia	10
5. Frontend y Alarmas en Tiempo Real	15
6. Grafana	10
7. Visualización Avanzada de Datos	10
8. HTTP vs. MQTT en IoT	5
9. Orquestación con Docker	10
10. Documentación	5
11. Preguntas	5
TOTAL	100

20.3 Detalle de la Calificación

En este apartado se utiliza la plantilla de rúbrica que debe tener obligatoriamente todo criterio de evaluación, con el rango de calificación Satisfactorio (100%-61%) y el rango Necesita mejorar (60%-0%).

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Punteo obtenido
1. Continuidad y Capa Física	**5**				
1.1	**Continuidad del nodo físico (Práctica 1)**	5	El nodo físico de la Práctica 1 opera correctamente e integrado al nuevo flujo (microcontrolador → backend por serial), sin alterar la lógica de control por umbrales. **Rango: 5 - 3**	El nodo físico no funciona correctamente, fue alterado sin justificación o no se integra con el backend. **Rango: 2 - 0**	
Sub-Total de Puntos	**5**				

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Punteo obtenido
2. Backend - API REST y Cliente MQTT	**15**				
2.1	**Lectura serial y publicación MQTT**	5	El backend lee de forma confiable el puerto serial y publica correctamente la telemetría y eventos en el broker MQTT. **Rango: 5 - 3**	Lecturas inconsistentes, pérdida de datos o publicación MQTT incompleta/no funcional. **Rango: 2 - 0**	
2.2	**Diseño y documentación de la API REST**	5	La API REST expone los endpoints necesarios, sigue buenas prácticas (verbos HTTP, códigos de estado) y está documentada (API Contracts). **Rango: 5 - 3**	Endpoints incompletos, mal diseñados o sin documentación. **Rango: 2 - 0**	
2.3	**Manejo de errores y reconexión**	5	El backend maneja adecuadamente la pérdida de conexión serial o MQTT, con reintentos y registro (logs) de eventos. **Rango: 5 - 3**	El backend se detiene o falla silenciosamente ante errores de conexión. **Rango: 2 - 0**	
Sub-Total de Puntos	**15**				
3. Broker MQTT	**10**				
3.1	**Jerarquía de topics, QoS, retained y LWT**	5	Se define y documenta una jerarquía de topics clara, con QoS adecuado, uso de retained messages y Last Will and Testament. **Rango: 5 - 3**	Topics desorganizados, sin documentación, o sin uso de QoS/retained/LWT. **Rango: 2 - 0**	
3.2	**Comunicación publish/subscribe funcional**	5	El broker distribuye correctamente los mensajes entre backend, Grafana y frontend, verificado con evidencia (cliente MQTT de prueba). **Rango: 5 - 3**	Fallos de comunicación o implementación incompleta de la suscripción/publicación. **Rango: 2 - 0**	
Sub-Total de Puntos	**10**				
4. Base de Datos y Persistencia	**10**				
4.1	**Modelo relacional**	5	El modelo entidad-relación es coherente con el dominio, normalizado y correctamente documentado. **Rango: 5 - 3**	Modelo incompleto, sin normalización adecuada o sin documentación. **Rango: 2 - 0**	
4.2	**Persistencia histórica**	5	Se conservan correctamente las lecturas, eventos y alarmas con su dimensión temporal, sin sobrescribir información previa. **Rango: 5 - 3**	No se conserva historial adecuado o los datos se sobrescriben incorrectamente. **Rango: 2 - 0**	

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Punteo obtenido
Sub-Total de Puntos	**10**				
5. Frontend y Alarmas en Tiempo Real	**15**				
5.1	**Visualización de datos y formularios**	5	El frontend visualiza correctamente los datos en tiempo real y mantiene los formularios/ficha de identificación heredados de la Práctica 1. **Rango: 5 - 3**	Visualización incompleta o formularios no funcionales. **Rango: 2 - 0**	
5.2	**Alarmas en tiempo real vía MQTT**	5	El frontend recibe y muestra alarmas en tiempo real mediante suscripción al broker (por ejemplo MQTT sobre WebSocket), de forma clara y visible. **Rango: 5 - 3**	Las alarmas no llegan en tiempo real, se implementan por polling/HTTP únicamente, o no son visibles. **Rango: 2 - 0**	
5.3	**Reportes PDF y usabilidad**	5	Se generan correctamente los reportes PDF y la interfaz es usable y responsiva en dispositivos móviles. **Rango: 5 - 3**	Reportes incompletos, no automáticos, o interfaz no usable en móviles. **Rango: 2 - 0**	
Sub-Total de Puntos	**15**				
6. Grafana	**10**				
6.1	**Dashboards en tiempo real (broker)**	5	Se implementan paneles conectados al broker MQTT que reflejan la telemetría en tiempo real. **Rango: 5 - 3**	Paneles no conectados en tiempo real o no funcionales. **Rango: 2 - 0**	
6.2	**Dashboards históricos (base de datos)**	5	Se implementan paneles de analítica histórica alimentados por consultas a la base de datos, con filtros de tiempo. **Rango: 5 - 3**	Paneles ausentes, sin filtros de tiempo, o consultas incorrectas. **Rango: 2 - 0**	
Sub-Total de Puntos	**10**				
7. Visualización Avanzada de Datos	**10**				
7.1	**Framework de visualización (Chart.js/D3/Plotly, etc.)**	4	Se integra un framework de visualización que consume datos reales del sistema mediante la API o el broker. **Rango: 4 - 2**	Framework no integrado o utiliza datos simulados/estáticos. **Rango: 1 - 0**	

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Punteo obtenido
7.2	**Visualización interactiva con p5.js o Three.js**	4	La visualización interactiva es creativa, funcional y reacciona correctamente a datos reales del sistema. **Rango: 4 - 2**	Visualización no interactiva, no funcional o desconectada de los datos reales. **Rango: 1 - 0**	
7.3	**Video demostrativo**	2	El video explica claramente la visualización interactiva y su relación con los datos del sistema. **Rango: 2 - 1**	Video ausente, incompleto o poco claro. **Rango: 0**	
Sub-Total de Puntos	**10**				
8. HTTP vs. MQTT en IoT	**5**				
8.1	**Documento comparativo y sustentación**	5	El documento explica correctamente ambos protocolos, incluye una tabla comparativa propia y justifica la elección de protocolos usada en el proyecto. **Rango: 5 - 3**	Comparación superficial, copiada, sin tabla propia o sin relación con las decisiones del proyecto. **Rango: 2 - 0**	
Sub-Total de Puntos	**5**				
9. Orquestación con Docker	**10**				
9.1	**docker-compose funcional**	6	El docker-compose levanta correctamente broker, backend y base de datos (idealmente también Grafana), comunicándose entre sí. **Rango: 6 - 3**	El docker-compose no funciona, faltan servicios obligatorios o no hay comunicación entre contenedores. **Rango: 2 - 0**	
9.2	**Buenas prácticas de orquestación**	4	Se utilizan redes, volúmenes para persistencia y variables de entorno para configuración sensible. **Rango: 4 - 2**	No se usan volúmenes, redes o variables de entorno adecuadamente. **Rango: 1 - 0**	
Sub-Total de Puntos	**10**				
10. Documentación	**5**				
10.1	**README completo**	5	El README incluye todos los elementos solicitados en la sección de Documentación, de forma clara y ordenada. **Rango: 5 - 3**	Documentación incompleta, desordenada o ausente en varios apartados. **Rango: 2 - 0**	
Sub-Total de Puntos	**5**				
11. Preguntas	**5**				

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Punteo obtenido
11.1	**Preguntas individuales**	5	Cada integrante del grupo contextualiza y responde correctamente las preguntas realizadas sobre el proyecto. **Rango: 5 - 1**	No se responde correctamente o se evidencia desconocimiento del trabajo realizado. **Rango: 0**	
Sub-Total de Puntos	**5**				
TOTAL	**100**				

20.4 Comentarios Generales

- Se evaluará el correcto funcionamiento integral de la plataforma IoT distribuida y la coherencia entre hardware, backend, broker, base de datos, Grafana y frontend.
- El proyecto debe demostrar comunicación MQTT estable, alarmas en tiempo real y separación clara entre las operaciones realizadas por REST y las realizadas por MQTT.
- Se valorará la calidad de la documentación técnica y la claridad en la explicación del proyecto, incluyendo el documento comparativo HTTP vs. MQTT.
- Todo el sistema deberá ser presentado funcionando en tiempo real durante la evaluación final, incluyendo la orquestación completa con Docker.